Human-in-the-loop loan approval agent: project plan

Oct 16, 2026 · @Alyan Ahmed Memon

Overview

We are building a loan approval agent where an AI analyst does the underwriting work and a loan officer steps in only where judgment is needed. The agent extracts the applicant's data, computes affordability ratios in code, and recommends approve, decline or refer, with each reason tied to a field or document line.

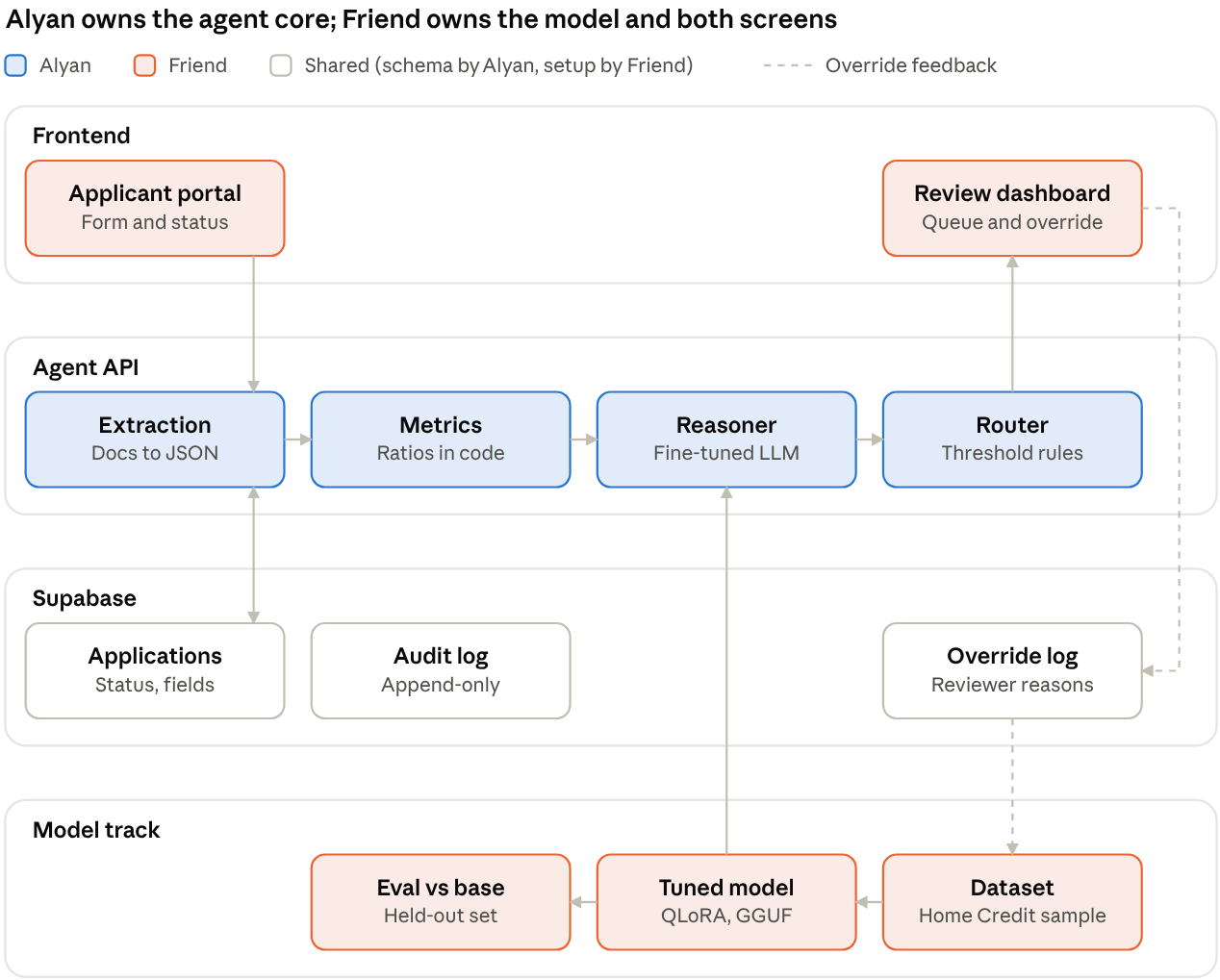
A rule-based router then sends each case one of three ways: auto-cleared, back to the applicant for missing information, or into a reviewer's queue. A person is pulled in for missing or conflicting data, borderline risk, loan amounts above a threshold, and every decline. Nothing is rejected without a human check.

Every action goes to an audit log, and every reviewer override is stored with its reason. We log overrides in this build but do not retrain on them; the table and export format are ready for a later round.

What we will show:

- A small fine-tuned LLM that explains policy-consistent decisions, evaluated against its base model on a held-out set.
- A case that pauses for a question and resumes after the applicant or reviewer answers.
- A reviewer override recorded with its reason.

System architecture



architecture · 4 tiers, 12 components

The agent API sits in the middle. Applications flow in from the portal, the router escalates to the reviewer, and the reasoner calls the tuned model through one endpoint.

Agent tools are exposed over MCP. Friend deploys the frontend, API and model to clouds.alternatefutures.ai.

The dashed path is the override loop: reviewer decisions land in the override log, which is the dataset for a later retraining round.

Who builds what

Alyan owns the logic: the API, the agent and everything the agent writes to the database. Friend owns the model, the frontend and everything that has to run in the cloud. If Friend is behind at hour 1.5, Alyan takes the applicant portal, since it is a single form page.

Workstream	Deliverable	Owner
Schema and contract	schema.sql, JSON contract, status enum, endpoint shapes	Alyan
FastAPI service	Create application, applicant reply and reviewer decision endpoints	Alyan
Agent pipeline	Extraction, metrics in code, reasoner call, router rules	Alyan
MCP server	Agent tools: get_application, compute_metrics, request_info, flag_for_review, log_event	Alyan
Audit and override logging	Append-only event log, override log with reviewer reasons	Alyan
Seed data and demo script	5 seeded applicants, one per demo path	Alyan
Container packaging	Dockerfile, requirements.txt, .env.example for the API	Alyan
Dataset	Home Credit sample, protected columns dropped, policy labels, generated rationales	Friend
Fine-tuning	QLoRA run, export, model served on an OpenAI-compatible endpoint	Friend
Evaluation	Base vs fine-tuned on a held-out set, one results table	Friend
Supabase setup	Project, schema applied, realtime on, storage bucket, keys	Friend
Frontend	Reviewer dashboard first, applicant portal second	Friend
Deployment	Frontend, API and model on clouds.alternatefutures.ai	Friend

Workstream	Deliverable	Owner
Pitch slides	Problem, architecture, evaluation table, fairness note	Friend

Build plan

The plan assumes about 4 hours; scale the slots if you have less. The two tracks run in parallel after the first 30 minutes, and the fine-tune is a drop-in swap behind one model endpoint, so neither of us waits on the other.

Slot	Step	Owner
0:00 to 0:30	Agree the schema, JSON contract, status enum and endpoint shapes; Alyan writes schema.sql and the contract file	Both
0:00 to 0:30	Create the Supabase project, accept the Kaggle competition rules and download application_train.csv, check what clouds.alternatefutures.ai can host (containers, model serving)	Friend
0:30 to 1:30	FastAPI skeleton, Supabase client, the three endpoints with a stubbed agent	Alyan
0:30 to 1:30	Dataset prep: drop protected columns, fix sentinel values, compute ratios, apply policy labels, generate rationales with an API model, spot-check about 50	Friend
1:30 to 2:30	Extraction, metrics, reasoner call against a prompted API model, router rules	Alyan
1:30 to 2:30	Start the QLoRA run (Unsloth, small Qwen3 or Llama model, 1 to 2 epochs on about 2k examples); build the reviewer dashboard while it trains	Friend
2:00	Hand over the Dockerfile and a working create-application endpoint; Friend makes the first deploy	Both

Slot	Step	Owner
2:30 to 3:00	MCP server and agent tools, audit and override logging, seed applicants	Alyan
2:30 to 3:00	Export and serve the tuned model, run base vs tuned on the held-out set, wire the dashboard to realtime	Friend
3:00	Cutoff: if the tuned model does not beat base on the eval, ship the prompted model and present the attempt as future work	Both
3:00 to 3:40	Integration: point the API at the model endpoint, run all seed cases end to end on the deployed stack; applicant portal	Both
3:40 to 4:00	Rehearse the demo and finish the slides; no new code	Both

Shared interfaces

These are the only things we both depend on, so they are locked in the first 30 minutes. Alyan owns them; Friend builds against them.

The reasoner returns this JSON, and the fine-tune is trained to produce exactly this shape:

```
{
  "recommendation": "approve | decline | refer",
  "confidence": 0.0,
  "reasons": ["short reason citing a field or metric"],
  "needs_human": true,
  "human_question": "specific question, or null"
}
```

The status flow is `submitted`, `analyzing`, then one of `needs_info`, `pending_review` or `auto_cleared`, and finally `decided`.

Interface	Owner	Consumer
POST /applications	Alyan	Frontend

Interface	Owner	Consumer
POST /applications/{id}/reply (applicant answers a question)	Alyan	Frontend
POST /applications/{id}/decision (approve, reject or override, with a reason)	Alyan	Frontend
Queue and status reads through Supabase realtime	Friend	Frontend
Model endpoint: OpenAI-compatible, set by base_url and model name in .env	Friend	Alyan

Until the tuned model is ready, the API points at a prompted API model. Swapping to the fine-tune changes one environment variable.

Risks and fallbacks

The fine-tune is the riskiest item, so the whole product is built first against a prompted API model. If any row below happens, the demo still works.

Risk	Fallback	Owner
Tuned model does not beat base by the 3:00 cutoff	Ship the prompted model; present the dataset, training run and eval attempt as future work	Friend
clouds.alternatefutures.ai cannot host the model	Serve the model elsewhere (for example a Hugging Face endpoint) or keep the prompted API model; decide in the first hour	Friend
Document extraction is unreliable	Drop uploads and run on form input only	Alyan
Kaggle download is blocked	Use German Credit for evaluation and the prompted model for the demo	Friend
Integration runs late	Freeze features at 3:00 and demo the seeded cases only	Both

One limit to state in the pitch: overrides are logged and exportable, but the model is not retrained on them in this build.

Demo plan

We run five seeded applicants live, each covering one path through the system. Alyan seeds the data and drives the demo; Friend presents the evaluation table and the architecture.

Applicant	Path	What it shows
Clean, small loan	auto_cleared	The agent works end to end without a human
Stated income differs from the bank statement	needs_info, then pending_review	The agent pauses with a specific question and resumes on the answer
Borderline risk	pending_review, reviewer overrides	The override and its reason land in the override log
Large loan amount	pending_review	A policy rule escalates regardless of score
Missing key fields	needs_info	The agent asks instead of guessing

We close on the evaluation slide: base vs fine-tuned on JSON validity, agreement with the policy label, and whether cited numbers match the inputs.